Независимый аудит Neuro-Kurisu 35.6.1 (архив B266 · HIDDEN_TASK_GENERALIZATION)

Проведён Claude, сентябрь 2026 · парный документ:
`BRAIN_BRANCH_CHAIN_PLAN_CLAUDE_RU.md`

---

0. Как читать этот документ и что в нём нового

Это не пересказ ваших собственных внутренних документов (`docs/brain/KARTA_ISKUSSTVENNOGO_MOZGA...`, `BRAIN_MASTER_PLAN_AND_AUDIT...`, `AUDIT_NEURO_KURISU_...B266...`). Эти документы я
прочитал полностью и они на голову выше среднего уровня подобных самоотчётов — местами честнее и строже, чем то, что писал бы я с нуля. Дублировать их бессмысленно.

Моя добавленная стоимость здесь — три вещи, которых не было в ваших документах:

1. **Независимая проверка кодом, а не чтением.** Я реально запустил ваши собственные тесты и аудит-скрипты (`node scripts/audit-*.mjs`, `node tests/*.mjs`) в изолированной среде — не поверил PASS/FAIL из документов на слово. Нашёл 1 реальную, воспроизводимую (3/3) регрессию, которой нет ни в одном прочитанном мной документе.
2. **Синтез поперёк времени.** Ваши документы честны в моменте (B262, B263… B266), но я сопоставил цифры МЕЖДУ проходами и увидел повторяющийся паттерн застревания, который не виден, если читать каждый аудит по отдельности.
3. **Внешнее сравнение.** Реальные живые AI-VTuber компаньоны (Neuro-sama и открытые клоны), классические когнитивные архитектуры (SOAR/ACT-R/LIDA), и текущая (2025-2026) научная литература по model-based RL — с конкретной, лично найденной научной статьёй, которая объясняет именно ту проблему, в которой вы сейчас находитесь (раздел 6).

---

1. Что это за архив — факты, не интерпретация

Это **не продолжение** телефон-приложения `v96work` (которое я аудировал ранее, версии v96.33→v99.80). Это отдельная, гораздо более зрелая

исследовательская ветка:

Параметр	v99.80 (предыдущий архив)	35.6.1 / B266 (этот архив)
package name	kurisu-neuro-mobile	neuro-kurisu-organism-evolution
Версия	99.80.0	35.6.1 (+ organismCore 70.0.0, state schem
Файлов всего	1251	2167
npm-зависимостей (dependencies)	9	**0**
npm-скриптов	422	**739** (из них 146 `audit:*`, 537 `test:*`)
Тестовых файлов	**0**	**516**
package-lock.json	нет	есть (v3, zero-dependency tree)
Архитектура	`v96work/src/neurocore/*` (249 файлов, много специалистов)	`src/organism/{core,cognition,memory` (~42 000 строк TS/MJS)
UI/чат-приложение	React/Vite/Express, Live2D в браузере	Отсутствует как таковое — `src/server/s`
Подключение к LLM	@google/genai SDK	Прямые `fetch()` к Gemini REST API + Ko

Важный вывод: «нулевая зависимость» — не поломка, а осознанное инженерное решение (`"packageLock": "v3-zero-dependency-tree"` в `VERSION.json`, `"runtimeDependencies": 0`). Это значит: код можно запускать **прямо из исходников через `node --experimental-strip-types`** без единого `npm install`. Я лично это проверил — Node 22.22.2 нативно исполняет `.ts`-файлы без транспиляции. Это прямое, реальное решение главного риска, который я находил в вашем телефон-архиве (OOM при `npm install` / `npm run build` в Termux) — **для этой ветки этот риск не существует в принципе**, если Termux даёт Node ≥22.

Архив назван в честь **последнего по времени большого прохода — B266 "Hidden Task Generalization"**, датированного 2026-09-12 (сегодня). Это буквально передний край разработки на момент вашего сообщения — не устаревший срез.

2. Независимая проверка: что я реально запустил и что нашёл

Я не поверил на слово ни одному "PASS" в ваших документах. Я лично выполнил:

```
node scripts/audit-core-ownership.mjs   → CORE_OWNERSHIP=PASS errors=0
node scripts/audit-architecture.mjs     → CLEAN_ARCHITECTURE=PASS
sourceFiles=966 canonicalFiles=349 cycles=0
node scripts/audit-source-hygiene.mjs   → SOURCE_HYGIENE files=1935
errors=0
node --experimental-strip-types scripts/audit-integrity.mjs →
CLEAN_INTEGRITY=PASS
node scripts/audit-forensics.mjs        → FORENSICS=PASS
node tests/big-pass-67-autobiographical-continuity.mjs → PASS (реальный
round-trip: flush→restart→restore, с настоящими assert)
```

Все — подтвердились. Это хорошая новость: базовая инженерная гигиена (нет дублей, нет циклических зависимостей, нет утечки legacy-кода в канонический путь, единое владение состоянием) — реальна, не декларативна.

2.1 Находка №1 (моя собственная, не из ваших документов): сломан `autoRun`

Я запустил полный `npm run audit:all` (весь исторический набор тестов проекта, не только brain-ветку) и он **упал**:

```
=== test:p15.5 ===
Error: autoRun did not advance organism cycle
    at tests/p15.5-device-hardening.mjs:45
AUDIT_ALL=FAIL stage=test:p15.5 code=1
```

Повторил 3 раза подряд — падает **стабильно, не флуктуирует** (то есть это не флейки-тест из-за таймингов, а реальная регрессия). Проследил причину до кода:

```
// OrganismRuntime.ts, строка ~548
decide: () => {
  const p = this.runtimeDecision();
  return { delayMs: p.nextDelayMs, reason: p.reasonCodes.join(','),
    run: p.cognitiveBudget01 >= 0.45 && this.embodiment.snapshot().awake
};
}
```

`autoRun: true` действительно запускает таймер-планировщик (`clockScheduler.start('cognitive')`) — сам таймер работает. Но условие `run:` внутри `decide()` требует `cognitiveBudget01 >= 0.45 && awake`. На свежесозданном организме (ровно то, что происходит при перезапуске процесса на телефоне) это условие, судя по всему, не выполняется — поэтому автономный тик **планируется, но не исполняется**.

Почему это важно для вашей цели, а не мелочь: `autoRun` — это буквально механизм, которым организм живёт **без внешнего сообщения от вас** — то самое «автономный независимый организм» из вашей формулировки задачи. Прямо сейчас, по факту, эмпирически проверено: организм не начинает жить сам, пока кто-то извне не подтолкнёт его достаточно, чтобы `cognitiveBudget01` пересёк порог 0.45. Это не теоретическая придирка — это конкретный, воспроизводимый регресс в самом ядре автономности.

Это не было упомянуто ни в одном из документов B259–B266, которые я читал — потому что все эти документы тестируют **brain-ветку в изоляции** (`test:big-pass-26X-core-integration`), а не через полный `audit:all`, который включает более старый `test:p15.5`. Это ровно тот тип проблемы, который синтез-по-всему-дереву находит, а точечный аудит одной ветки — нет.

2.2 Находка №2: документационная гигиена в корневом README

`README_RU.md` содержит **10 разных строк «Current release: X»**, наслоенных друг на друга без чистки, со **state schema номерами** 104, 110, 120, 121, 122, 123, 127 вперемешку — то есть человек, открывший файл, не может с первого взгляда понять, какая версия актуальна, не читая весь файл целиком в поисках самой последней строки. Это не критично функционально, но это ровно тот тип «декоративного беспорядка» (документация, которая перестала быть источником истины), о котором вы просили не забывать. Хорошая новость: `VERSION.json` и `RELEASE_MANIFEST.json` **не имеют** этой проблемы — они чистые, единственные, машиночитаемые источники правды. Рекомендация: README должен **ссылаться** на них, а не дублировать номер версии текстом.

---

3. Синтез поперёк времени: где проект реально «застревает»

Здесь я использую данные из ваших собственных документов (не выдумываю), но сопоставляю их так, как не сделал ни один отдельный аудит, потому что каждый из них смотрит только на «предыдущий шаг → этот шаг».

3.1 Паттерн №1 — ширина растёт быстрее глубины доказательств

`FUNCTION_REALITY_STATUS_180.json` (пройден до brain-ветки B207+): из **2493** отслеживаемых функций только **38 (1.5%)** имеют статус `VERIFIED` (прямое + регрессионное + persistence/replay доказательство), **1641 (65.8%)** — `UNVERIFIED`. Дельта между проходом 179 и 180: **0 изменений** — то есть за целый большой проход верификация не сдвинулась, пока количество кода продолжало расти. Это количественное, не оценочное подтверждение риска «декорации» — не с моих слов, а из вашего собственного трекинга.

3.2 Паттерн №2 — каждый новый слой причинности пока даёт около-нулевое поведенческое улучшение

Прямая цитата чисел из ваших же аудитов, выстроенных в ряд:

Проход	Что добавлено	Поведенческий результат
B262	Multi-timescale world model + beam planner	`planningAdvantage01 = -0.1743` (планировщик **хуже**, чем без него)
B263	Real action-consequence causality	`behavioralGain01 = 0`
B264	Causal policy learning	(частично положительный, но с «self-evaluation loophole», исправленным только в B265)
B265	Независимый оценщик (не доверяет self-report)	Core advantage «positive but modest (+0.0513)»; evaluator agreement в Core ~0.486 (то есть слабое согласие)
B266	Обобщение на скрытые семейства задач	train gain 0.035, hidden-family gain 0.028 — положительно, но малые абсолютные величины

Это не хаотичный список неудач — это последовательное, честное приближение к одной и той же стене, и сам проект абсолютно точно называет эту стену в разделе 10 аудита B262: `action diversity → measurable consequence diversity → identifiable causal model` — то есть у организма пока слишком мало **реально различных** последствий разных действий, поэтому causal-модели физически не из чего учиться отличать хорошее действие от плохого. Каждый новый B-проход добавляет более изощрённый механизм ИЗМЕРЕНИЯ этой проблемы, но не решает её первопричину — недостаток разнообразного embodied-опыта.

3.3 Паттерн №3 — общепроектная регрессионная целостность отстаёт от глубины отдельной ветки

Brain-ветка (B259-B266) регрессионно чиста сама по себе (я проверил). Но весь проект целиком (`audit:all`, включающий P7-P15.5 и далее) — **нет** (находка 2.1). Это тот же паттерн, что и 3.1, но на уровне инженерии, а не науки: глубокая работа на переднем крае идёт быстрее, чем поддержание чистоты всего дерева целиком.

Синтез: все три паттерна — это ОДИН и тот же более общий паттерн: **проект систематически предпочитает открывать новый фронт вместо того, чтобы закрыть текущий до конца**. Сам проект отчасти это уже осознал — раздел 11 аудита B266 прямо говорит: «главный разрыв находится уже не между модулями, а между evaluation и self-directed adaptation» — то есть проект сам движется в сторону признания этого паттерна. Мой вклад здесь — показать, что этот же паттерн повторялся минимум 3 раза на разных уровнях (верификация функций, поведенческая причинность, общепроектная регрессия), а не один раз случайно.

---

4. Оценки 1–100 — сведены воедино (ваши данные + независимая перепроверка)

Я **не изобретаю** параллельную систему оценок с нуля — я беру ваши собственные (строгие, честные) оценки из B265/B266 как основу, подтверждаю их независимой проверкой там, где мог, и добавляю несколько измерений, которых не было ни в одном из ваших внутренних аудитов.

4.1 Из вашего B266-аудита (подтверждено чтением исходников, где указано «проверено Claude»)

Элемент	Оценка	Проверено Claude?
Persistence	97	✅ да (round-trip тест запущен лично)
Migration discipline	96	частично (чтение кода миграций)
Release engineering	94	✅ да (архитектурный/hygiene/forensics аудит запущен лично)
Independent evaluation	72	частично (чтение `B265IndependentPolicyEvaluator.ts`)
Causal policy	66	нет (доверие вашему числу)

Элемент	Оценка	Проверено Claude?
Brain branch engineering	86	✅ да, совпадает с моим впечатлением от чтения кода
Hidden-family generalization	58	нет (доверие вашему числу)
Overall brain scientific maturity	66	нет (доверие вашему числу)

4.2 Новые измерения, добавленные Claude (не было в ваших brain-аудитах)

Элемент	Оценка 1–100	Обоснование
Автономность запуска без внешнего триггера (autoRun)	**15**	Эмпирически: тест падает 3/3, механизм спланирован, но гейт не пропускает исполнение на свежем организме
Регрессионная целостность всего дерева (не только brain-ветки)	**≈60**	`audit:all` падает на первом же более старом тесте (p15.5); неизвестно, сколько ещё скрытых регрессий дальше по цепочке — я не прогонял весь список из 537 тест-скриптов до конца из-за бюджета времени, честно отмечаю неопределённость
Документационная гигиена (README как источник истины)	**35**	10 наслоенных «Current release» строк, схема версий 104-127 вперемешку в одном файле
Готовность к реальному телефону (Termux/Android) именно для ЭТОЙ ветки	**72**	Ноль зависимостей = нет риска OOM при install/build (сильно лучше, чем в v99.80); НО отсутствует прямая проверка на физическом Android-устройстве («Android device gate» сам проект честно помечает как невыполненный)
Честность/anti-decoration дисциплина как процесс	**93**	Function Passport, Unified Proof Protocol, gap ledger, явный запрет «не считать функцию реализованной до прохождения proof tier» — это одна из лучших такого рода систем, что я видел в подобных проектах

Элемент	Оценка 1–100	Обоснование
Связность речи с решением (LLM как «языковой орган», не источник решений)	**80**	`LanguageOrgan.ts` явно инструктирует модель не изобретать внутреннее состояние и не менять уже принятое решение — архитектурно верно; не проверял эмпирически, действительно ли LLM это соблюдает на практике (нужен отдельный adversarial-тест: попытаться заставить LLM «сломать» инструкцию)

4.3 Прогресс по вашей собственной иерархии причинной цепочки (из B265, раздел 12), в процентах

```
Observation                       100%
P27 latent                         100%
B259 recurrent learning             82%
B260 delayed memory                 76%
B261 world model                    77%
B262 multi-timescale dynamics       73%
B263 action causality               68%
B264 causal policy                  69%
B265 independent evaluation         72%
B266 hidden-family generalization  ~65% (моя оценка на основе
прочитанных чисел; ваш документ B266 не дал отдельного % в этом формате)
long-horizon behavioural superiority 28%
real external action                25%
autoRun / автономный тик без триггера ~15% (моя новая находка, не было в
цепочке)
```

Обратите внимание на форму этой кривой: она **монотонно убывает** от 100% (наблюдение) до 25% (реальное внешнее действие) почти без исключений — то есть чем ближе к «настоящему воплощённому поведению в реальном мире», тем меньше доказанности. Это ожидаемо и честно, но важно видеть это как ЦЕЛЬНУЮ кривую, а не набор изолированных чисел.

---

5. Что сделано особенно хорошо (без преувеличения)

1. **Zero-dependency подход** — устраняет главный риск предыдущей архитектуры (OOM на телефоне) полностью, а не частично.
2. **Function Passport / Unified Proof Protocol** — формализация Anti-Decoration принципа глубже, чем я предлагал в прошлый раз: разделение на direct/regression/replay/persistence/longitudinal/performance/

failure/acceptance доказательства по классам функций — это методологически зрелее многих академических cognitive-architecture проектов, которые я нашёл при сравнении (раздел 6).

3. **Готовность признавать отрицательный результат** — `planningAdvantage01 = -0.1743`, `behavioralGain01 = 0` — это честность высочайшего уровня, крайне редкая даже в опубликованных исследованиях, не то что в персональных AI-проектах.
4. **`lesion()` / `rescueLastLesion()`** в `P31FunctionalConsciousness.ts` — реальный, вычислительно содержательный ablation-механизм (не декорация — я лично прочитал формулу: конкретные множители на конкретные метрики в зависимости от режима).
5. **516 реальных тестовых файлов с настоящими assert** (я лично запустил и проверил один построчно) — качественный скачок от 0 тестов в предыдущем архиве.
6. **Языковой слой архитектурно подчинён когнитивному ядру**, а не наоборот — `LanguageOrgan.ts` явно требует от LLM не выдумывать состояние и не менять уже принятое решение.

5.1 Что сделано плохо (без преувеличения)

1. **`autoRun` не работает** — см. находку 2.1. Это P0.
2. **Полный `audit:all` не проходит** — неизвестный объём скрытых регрессий за пределами brain-ветки.
3. **README как источник правды сломан** документационным нагромождением.
4. **65.8% функций всё ещё `UNVERIFIED`** на последнем известном срезе (пройдено 180, до brain-ветки — сейчас может быть иначе, но нет данных, что это принципиально улучшилось, поскольку более новые аудиты B259-266 не публиковали общепроектный `FUNCTION_REALITY_STATUS` заново — стоит перепроверить).
5. **Основной причинный бутылочный узел (action diversity → consequence diversity) не решён за 5 больших проходов подряд** (B262-B266) — не потому что решения не пробовали, а потому что первопричина (недостаток embodied-разнообразия) требует другого типа работы (раздел 7), а не ещё одного слоя мат. модели поверх той же скудной среды.

---

6. Внешнее сравнение — что можно позаимствовать

6.1 Живые AI-компаньоны (Neuro-sama и открытые клоны)

<cite index="8-1">Открытый клон Neuro-Sama устроен подчёркнуто модульно: разделён на независимые файлы (TTS, распознавание речи, обёртка LLM, модуль решения, когда и как обращаться к LLM), где все модули читают и пишут в один общий объект сигналов, а память/RAG автоматически генерируется по мере разговора и сохраняется между перезапусками</cite>. Управление Live2D-аватаром реализовано просто — через горячие клавиши и предустановленные позиции, без попытки связать каждое движение с внутренним «сознанием».

Вывод для вас: Neuro-sama (оригинал) стримит **годами**, стабильно, с миллионами подписчиков — не потому что у неё под капотом causal world model с ablation-экспериментами, а потому что простой, тонко отполированный, специализированный пайплайн с низкой задержкой оказался достаточен для убедительного «ощущения жизни» у зрителя. Это не довод против вашей архитектуры (ваша цель принципиально другая — не развлекательный стрим, а попытка реальной когнитивной глубины) — но это важный контрапункт: **глубина архитектуры и убедительность «живости» для внешнего наблюдателя — разные оси**, и можно застрять, оптимизируя одну в ущерб другой. Возможно, стоит сознательно выделить отдельный, более простой «fast path» для обычного разговорного взаимодействия (похожий на архитектуру клона Neuro), не пропуская каждую реплику через полный B259–B266 causal stack — и резервировать глубокий стек для случаев, когда `cognitiveBudget01` и новизна ситуации это оправдывают. Похоже, зачатки этого у вас уже есть (Tier 0–4 resource-based compute-budget в мастер-плане) — просто явно скажите себе, что это осознанный, а не вынужденный компромисс.

6.2 Классические когнитивные архитектуры (SOAR/ACT-R/LIDA)

<cite index="15-1">Когнитивные архитектуры определяются как спецификации, описывающие структурные элементы и возможности, необходимые для когнитивной системы, включая обучение, рассуждение, принятие решений, восприятие, планирование и действие</cite> — то есть буквально тот же список доменов, что и в вашей карте `src/organism`. Это не совпадение — вы (через Grok) фактически заново открываете задачи, которые эта область решает 40+ лет.

Что конкретно можно позаимствовать у SOAR — **impasse-driven subgoaling**: когда система не может решить, что делать (несколько конкурирующих правил активны одновременно, или ни одно не активно), SOAR автоматически создаёт **подцель** — «разрешить это противоречие» — вместо того, чтобы просто выбрать что-то по умолчанию или зависнуть. Применимо к вашему `cognitiveBudget01`-гейту: вместо того, чтобы просто **не исполнять** тик, когда бюджет недостаточен (текущее поведение, приводящее к найденному мной багу автономности), система могла бы **создавать явную под-цель** «восстановить бюджет» или «понять, почему

бюджет низкий» — то есть отсутствие ресурса становится **содержанием мысли**, а не тихим пропуском цикла. Это одновременно чинит найденный баг и делает его починку концептуально интереснее, чем просто «понизить порог с 0.45 до 0.2».

6.3 Научная литература: ваша текущая проблема имеет имя

Это, вероятно, самая практически полезная находка раздела. <cite index="25-1">B model-based обучении с подкреплением возникает так называемое несоответствие целей (objective mismatch): цели, максимизируемые при обучении политики и при обучении модели мира, не согласованы и не полностью разделимы, потому что политика стремится максимизировать будущую награду, а модель мира — минимизировать ошибку предсказания переходов среды безотносительно к задаче</cite>. Это ровно то, с чем вы столкнулись в B262 (хорошее предсказание, отрицательное planning advantage): **точная модель мира не гарантирует полезную для политики модель мира** — это не ваша инженерная ошибка, это фундаментальное, активно исследуемое свойство самого подхода.

Более того, <cite index="24-1">даже очень точные модели могут не давать сильных политик — обучение с подкреплением внутри выученных моделей мира испытывает трудности как с производительностью, так и с масштабируемостью именно из-за рассогласования между улучшением политики и обучением функции ценности</cite>.

Конкретные, уже опубликованные направления исправления, которые стоит изучить для B267+ (я специально не выдаю это как готовое решение "под ключ" — это направления для вашего/Grok исследования, не патч):

- Обучать модель мира с **task-aware** функцией потерь (не просто минимизировать ошибку предсказания состояния, а явно учитывать, что предсказание полезно именно для различения хороших/плохих действий) — это прямая, более научно обоснованная альтернатива тому, что ваш B262 уже интуитивно нащупал через `stateDistance01` / `rewardDelta01`.
- Rollout данные для обучения политики собирать так, чтобы явно поощрять **разнообразие действий** (то, что вы сами назвали корнем проблемы) — а не просто увеличивать точность предсказания на существующем, малоразнообразном распределении действий.

---

7. Моя собственная гипотеза — новый метод, которого нет ни в одном из ваших документов

Все пять проходов (B262–B266) пытаются решить проблему «мало разнообразных последствий действий» **добавлением более умной**

математики поверх существующего скудного взаимодействия со средой. Ваш собственный B262-аудит уже почти сформулировал это правильно («расширить реальный sensorimotor consequence space», не «искусственно увеличивать чувствительность») — но ни один последующий проход (263, 264, 265, 266) фактически **не расширил среду**, а продолжил улучшать измерительный аппарат вокруг той же среды.

Предлагаю (метод, придуманный мной для этого случая): Curiosity-Gated Action Diversification Buffer. Идея: вместо пассивного ожидания, пока разнообразные действия появятся сами по себе из обычного диалога с пользователем, ввести отдельный, явно помеченный «экспериментальный» режим (запускаемый только при достаточном `cognitiveBudget01` и явном consent-флаге пользователя — не тайно), в котором организм **намеренно и явно** выбирает наименее опробованные из безопасного набора действий/реакций в текущем контексте, специально ради того, чтобы причинная модель получила разнообразные примеры «действие → исход». Ключевое отличие от классического RL-exploration (epsilon-greedy и т.п.): здесь разнообразие собирается **офлайн, в структурированных эпизодах** (аналог целенаправленной игры/исследования, а не случайного шума в реальном разговоре с вами) — то есть организм в отдельном, явно обозначенном «времени для экспериментов» сам придумывает мини-сценарии для себя (текстовые, раз внешних сенсоров пока нет) и проверяет на них разные действия, специально чтобы разнообразить `B263RealActionCausality` / `B264CausalPolicyLearning` данные, не трогая качество обычного разговора с вами. Это прямое, конкретное дополнение к плану B267 (раздел 8 паре-документа), которое атакует первопричину, а не очередной измерительный слой поверх неё.

---

8. Итоговый вердикт

Проект прошёл огромный путь с версии, которую я аудировал в августе (v99.80, телефонное приложение с реальными, но простыми и местами декоративными модулями) до этой исследовательской ветки — на порядок серьёзнее по инженерной и научной дисциплине. Anti-decoration принцип, который я формулировал как совет в прошлый раз, теперь реализован **в коде и в процессе** (Function Passport, Unified Proof Protocol) лучше, чем я мог бы это описать словами.

Главный риск сейчас — не декорация в старом смысле (стаб вместо 64 модулей), а **системное застревание на одном и том же уровне абстракции** (ещё один слой измерения причинности) вместо перехода на следующий уровень (реальное разнообразие взаимодействия), и **скрытая регрессия автономности** (`autoRun`), которая прямо противоречит

центральной цели проекта и которую не поймал ни один из 6 подряд brain-специфичных аудитов, потому что все они тестировали ветку в изоляции от целого дерева.

Продолжение — в `BRAIN_BRANCH_CHAIN_PLAN_CLAUDE_RU.md`.